Linear-time sorting solutions

08-E01 Comparison lower bound

A height-h binary tree has at most 2^h leaves. Distinguishing 24 permutations requires $2^h \geq 24$. Since $2^4=16<24$ and $2^5=32 \geq 24$, the worst case requires at least five comparisons. Counting sort also reads integer values as count-table indices, so it is outside this comparison-only model.

08-E02 Stable counting placement

Frequencies are [1,1,3], and cumulative endpoints [1,2,5]. Scan backward: 2e goes to position 5, reducing endpoint 2 to 4; 1d goes to 2, reducing endpoint 1 to 1; 2c goes to 4; 0b to 1; 2a to 3. Output is [(0,b),(1,d),(2,a),(2,c),(2,e)]. Later equal records take farther-right positions first; earlier ones take farther-left positions, preserving a,c,e.

08-E03 Radix cost tradeoff

The modeled costs are 4(n+256)=4n+1024 and 2(n+65536)=2n+131072. At equal unit costs, the second is smaller iff 2n>130048, or n>65024. Eight-bit digits have smaller initialization overhead; sixteen-bit digits need fewer passes. This is a modeled crossover rather than measured timing. Fewer passes alone do not prove a win because each table initializes r entries. Memory, cache, key distribution, stable placement costs, and language implementation also matter.

08-E04 Exactly where stability fails

Start C=[1,2,5]. Forward placement puts 2a at 5,0b at 1,2c at 4,1d at 2,2e at 3. Output [0b,1d,2e,2c,2a] is key-sorted but reverses equal identities. On [12,11], the units pass produces [11,12]; reversing equal tens in the next pass gives the incorrect [12,11].

Backward scanning is stable with decrementing endpoints. Forward scanning can be stable with incrementing starts. LSD induction specifically needs each current-digit group to preserve the earlier lower-digit order, which stability supplies.

08-E05 Nonuniform bins and squared sizes

Expand ordered same-bin record pairs:

$$\sum_i N_i^2 = n + \sum_{j \neq k} I[j, k \text{ in same bin}].$$

Independence makes pair probability $\Sigma p_i^2=1/4+1/16+1/16=3/8$. Hence expectation is $4+4 \cdot 3 \cdot 3/8=17/2$. Uniform bins give $4+12/4=7$.

Linear-time sorting solutions (continued)

For a dependent counterexample, choose J uniformly in 0..3 and put all four records in J. Each bin still has expected size 1, but squared sizes always sum to 16. $E[N_i^2]$ cannot be replaced by $(E[N_i])^2$, nor record independence by uniform marginals. This concerns the squared-work bound; equal values themselves do not force quadratic stable-insertion time.

08-E06 Prefix queries and negative-key shifts

Map key v to v+2. In order −2,−1,0,1,2,3, frequencies are [2,0,1,2,0,1], with saved prefixes P=[2,2,3,5,5,6]. Range [−1,1] has P[3]−P[0]=5−2=3 records. Range [−2,−2] has P[0]−P[−1]=2−0=2, using boundary convention P[−1]=0.

Each output placement decrements C[key], destroying its original ≤key total. Copy P before placement. Preprocessing costs Θ(n+range size), each query O(1); shifting does not compress a huge sparse range.

Verification code appendix: counting_sort.py

```
"""Stable counting sort for nonnegative integer keys."""

def counting_sort(values, key_max):
    counts = [0] * (key_max + 1)
    for value in values:
        counts[value] += 1
    for i in range(1, len(counts)):
        counts[i] += counts[i - 1]
    result = [0] * len(values)
    for value in reversed(values):
        result[counts[value] - 1] = value
        counts[value] -= 1
    return result


sample = [2, 0, 2, 1, 2]
assert counting_sort(sample, 2) == [0, 1, 2, 2, 2]
print(counting_sort(sample, 2))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Linear-time sorting solutions (continued)

Verification code appendix: linear_sort_checks.py

```
"""Stable identity traces and exact finite bucket-distribution calculations."""
from itertools import product
from fractions import Fraction


def counting(records, key_max, key=lambda record: record[0], reverse=True):
    counts=[0]*(key_max+1)
    for record in records:
        counts[key(record)]+=1
    frequencies=counts[:]
    for i in range(1,len(counts)):
        counts[i]+=counts[i-1]
    prefix=counts[:]
    output=[None]*len(records)
    placements=[]
    indices=range(len(records)-1,-1,-1) if reverse else range(len(records))
    for j in indices:
        value=key(records[j])
        slot=counts[value]-1
        output[slot]=records[j]
        counts[value]-=1
        placements.append((j+1,records[j],slot+1,counts[:]))
    return output,frequencies,prefix,placements


def radix(records,base,digits):
    values=list(records)
    states=[values[:]]
    power=1
    for _ in range(digits):
        values=counting(values,base-1,key=lambda r:(r[0]//power)%base)[0]
        states.append(values[:])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear-time sorting solutions (continued)

```
        power*=base
    return states


def insertion(values):
    result=list(values)
    moves=0
    for j in range(1,len(result)):
        value=result[j]
        i=j-1
        while i>=0 and result[i][0]>value[0]:
            result[i+1]=result[i]
            i-=1
            moves+=1
        result[i+1]=value
    return result,moves


def bucket(records):
    n=len(records)
    if not n:
        return [],[],0
    buckets=[[] for _ in range(n)]
    for record in records:
        assert 0<=record[0]<1
        index=(n*record[0]).numerator//(n*record[0]).denominator
        buckets[index].append(record)
    sorted_buckets=[]
    moves=0
    for group in buckets:
        ordered,cost=insertion(group)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear-time sorting solutions (continued)

```
        sorted_buckets.append(ordered)
        moves+=cost
    return [r for group in sorted_buckets for r in group],sorted_buckets,moves


def main():
    checked=0
    for n in range(8):
        for keys in product(range(3),repeat=n):
            records=list(zip(keys,range(n)))
            ordered=counting(records,2)[0]
            assert ordered==sorted(records,key=lambda r:r[0])
            checked+=1
    sample=[(2,'a'),(0,'b'),(2,'c'),(1,'d'),(2,'e')]
    ordered,freq,prefix,placements=counting(sample,2)
    assert freq==[1,1,3] and prefix==[1,2,5]
    assert [p[2] for p in placements]==[5,2,4,1,3]
    assert counting(sample,2,reverse=False)[0]==[(0,'b'),(1,'d'),(2,'e'),(2,'c'),(2,'a')]
    values=[412,205,421,115,212,405]
    states=radix(list(zip(values,range(6))),10,3)
    assert [[r[0] for r in s] for s in states]==[values,[421,412,212,205,115,405],
[205,405,412,212,115,421],[115,205,212,405,412,421]]
    radix_cases=0
    for n in range(5):
        for keys in product([0,1,3,4,11,15],repeat=n):
            records=list(zip(keys,range(n)))
            for base,digits in [(2,4),(4,2),(10,2)]:
                assert radix(records,base,digits)[-1]==sorted(records,key=lambda r:r[0])
            radix_cases+=1
    assignments=0
    for n in range(1,6):
        square_total=0
        for cells in product(range(n),repeat=n):
```

Linear-time sorting solutions (continued)

```
            sizes=[cells.count(i) for i in range(n)]
            square_total+=sum(size*size for size in sizes)
            assignments+=1
        assert Fraction(square_total,n**n)==2*n-1
    probabilities=[Fraction(1,2),Fraction(1,4),Fraction(1,4),Fraction(0)]
    expected=Fraction(0)
    for cells in product(range(4),repeat=4):
        probability=Fraction(1)
        for cell in cells:
            probability*=probabilities[cell]
        expected+=probability*sum(cells.count(i)**2 for i in range(4))
    assert expected==Fraction(17,2)
    for n in range(2,40):
        descending=[(Fraction(n-j,2*n*n),j) for j in range(n)]
        ordered,groups,moves=bucket(descending)
        assert ordered==sorted(descending,key=lambda r:r[0])
        assert len(groups[0])==n and moves==n*(n-1)//2
    bucket_cases=0
    for n in range(7):
        for keys in
product([Fraction(0),Fraction(1,5),Fraction(1,2),Fraction(9,10)],repeat=n):
            records=list(zip(keys,range(n)))
            assert bucket(records)[0]==sorted(records,key=lambda r:r[0])
            bucket_cases+=1
    assert 4*65024+1024==2*65024+131072
    print(f"PASS stable_count_arrays={checked} radix_arrays={radix_cases}
exact_uniform_bucket_assignments={assignments} nonuniform_square_moment={expected}
bucket_arrays={bucket_cases} worst_descending_bucket=True")


if __name__=="__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study